

AsCode Solution

Inventory Management & POS System

Full Technical Design & Development Specification

Offline-Installable Windows Edition

Client: Mr. Abdullah | Prepared By: AsCode Solution | Deployment: Single Local Store, On-Premise, Offline-First

This document is written to be handed directly to a development agent (e.g. Claude Code) as the single starting brief for this project. It defines the product, the system design, the technology stack and the reasoning behind every major choice, the trade-offs considered, the phased build plan, and the exact rules the agent must follow — including when it must stop and ask for human approval before proceeding.

Table of Contents

(This field populates automatically when the document is opened in Microsoft Word. If it appears empty, right-click it and choose "Update Field".)

0. How to Use This Document (Read First)

This is the full development brief for the project. It is written so that an autonomous or semi-autonomous coding agent can read it once and begin work without needing a separate kickoff conversation. It is organized so that:

- Section 1 defines a mandatory process skill the agent must use throughout the project.
- Section 2 defines the non-negotiable operating rules ("laws") the agent must follow.
- Sections 3–6 define the product, system design, stack, and reasoning behind each choice.
- Section 7 defines every feature and why it exists.
- Section 8 records the key trade-off decisions already made, and why.
- Section 9 defines the living documentation system the agent must maintain.
- Section 10 is the edge-case and test matrix that must be honoured throughout.
- Section 11 defines the testing/QA strategy and how the agent proves a phase is actually done.
- Section 12 is the phase-by-phase build plan — the actual execution order.
- Section 13 is the final "ready for production" definition of done.
- Section 14 is the literal first-session checklist for the agent to follow, starting now.

Wherever this document marks a decision as recommended rather than fixed, the agent should proceed with the recommendation by default but must still surface it explicitly at the relevant STOP AND ASK gate before it becomes irreversible (e.g. before a schema migration is frozen, or before packaging choices are baked into the installer).

1. Mandatory Process Skill — grill-with-docs

Before any phase that materially shapes the architecture, data model, or an external integration, the agent must invoke the grill-with-docs skill. This is not optional tooling — it is the mechanism by which this project's design gets pressure-tested and documented as it goes.

```
---
name: grill-with-docs
description: A relentless interview to sharpen a plan or design, which also creates docs (ADR's and
glossary) as we go.
disable-model-invocation: true
---
```

Because disable-model-invocation is set, the agent cannot trigger this skill automatically — a human must invoke it. The agent's job is to tell the human when it is time to run it, and to refuse to treat the relevant design area as final until that grill session has happened and its outputs (ADR + glossary updates) exist in /docs.

When it must be used

- Phase 0 — before the data model and module boundaries are frozen.
- Phase 5 — before the WhatsApp provider, message templates, and retry policy are chosen.
- Phase 7 — before the Windows packaging, auto-start, and update-strategy decisions are locked.
- Any time the agent or the human proposes a design change that would alter an existing ADR.

What it produces

- New or updated entries in /docs/adr/ (one ADR per real decision).
- New or updated terms in /docs/glossary.md.

- A sharper, interrogated version of whatever plan was put in front of it — contradictions, unhandled edge cases, and vague requirements surfaced before code is written, not after.

2. Operating Rules for the Coding Agent

These rules apply for the entire project, across every phase. They are not suggestions.

1. Never skip a scheduled **grill-with-docs** session — proceeding without it on a flagged decision is a rule violation, not a shortcut.
2. **Never proceed past a  STOP AND ASK gate** without explicit human approval recorded in context.md. Present the decision, state the recommendation, then wait.
3. Every irreversible or costly-to-reverse technical decision gets an **ADR** written before it is implemented, not after.
4. No feature is "done" without passing tests **and** an updated context.md entry. Code without tests is unfinished code.
5. Tests are written **alongside** the feature, not after it. A phase cannot close with red, skipped, or missing tests.
6. Never hardcode secrets, API keys, or credentials. All configuration comes from environment variables / a local .env that is never committed.
7. Every database schema change goes through a migration. Manual, undocumented schema edits are forbidden.
8. All money and quantity values use fixed-point/integer (e.g. integer paisa/cents) handling — never raw floats — to eliminate rounding bugs in billing and ledgers.
9. All external calls (WhatsApp, or any future API) must be **non-blocking** to the core sale flow. A sale must be able to complete even if the external service is down.
10. On a flagged decision, the agent recommends the best option and states its reasoning — it does not silently choose on the human's behalf.
11. Business logic stays out of route handlers. Keep functions and modules small, single-purpose, and unit-testable in isolation from Flask.
12. Every bug fix ships with a regression test, not just a patch.
13. No feature creep beyond the scoped module list without a new ADR and explicit human approval.
14. The glossary is updated the moment a new domain term is introduced, not retroactively.

3. Product Overview

A single-store Inventory Management & POS System, built with Python/Flask and SQLite, that runs entirely on a PC inside the shop and is reachable by other in-shop terminals over the shop's own network. The system now also ships as a Windows-installable, offline-capable desktop application — not just a browser-accessed local web app.

Product Goals

- Let a cashier complete a sale — cash, card, or credit — in seconds, using barcode/QR scanning wherever possible.
- Keep inventory numbers correct automatically, without manual reconciliation after every sale.
- Track credit customers accurately and notify them on WhatsApp without ever slowing down the till.
- Work fully offline for all core operations; only WhatsApp notifications require internet.

- Install and start like a normal Windows program on the server PC — no command line, no "start the server manually" step for daily use.
- Be maintainable and testable by a coding agent working largely unsupervised, with clear checkpoints for human judgment calls.

Explicitly Out of Scope (this phase of the product)

- Multi-store / multi-branch synchronization.
- Full configurable RBAC (role/permission matrix beyond Admin/Cashier).
- Cloud hosting, remote access, or a public-facing storefront.
- Native mobile apps.

4. System Design

4.1 System Architecture Diagram

One PC in the shop runs the installed application (Flask app packaged with PyInstaller, wrapped in a PyWebview desktop window, auto-starting on login). Other terminals reach it over the LAN through a browser — no install needed on those. The barcode scanner and thermal printer are ordinary LAN-local hardware. The only component that needs the internet is the WhatsApp notification path, and it is architected so its absence never blocks a sale.

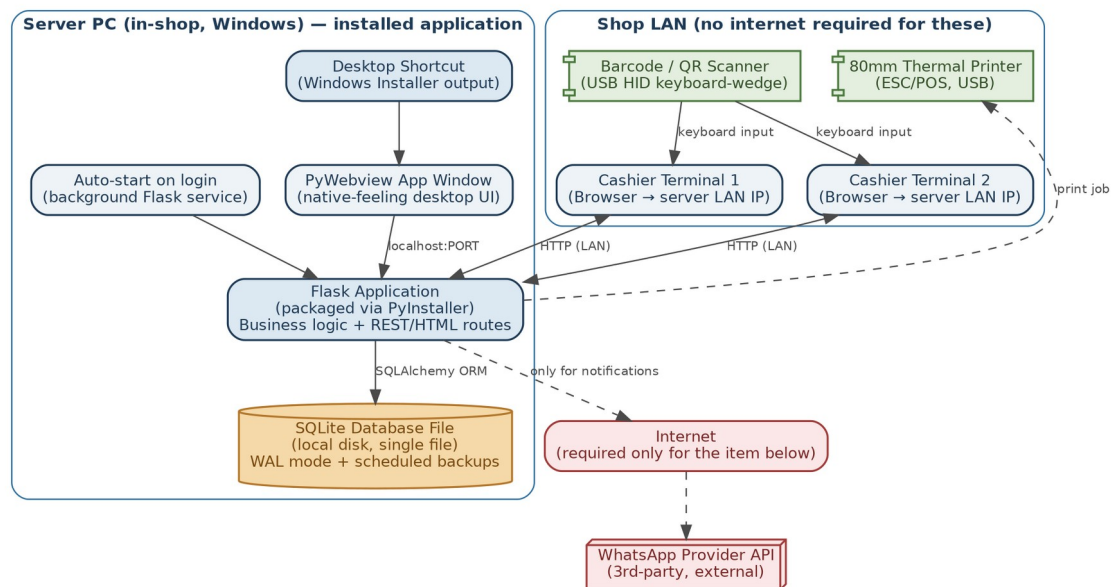


Figure 1 — System architecture: server PC, LAN terminals, hardware, and the isolated internet dependency.

4.2 Process / Data Flow Diagram

This is the core business journey the system exists to support — from product creation through to a completed, recorded, and (if applicable) notified sale.

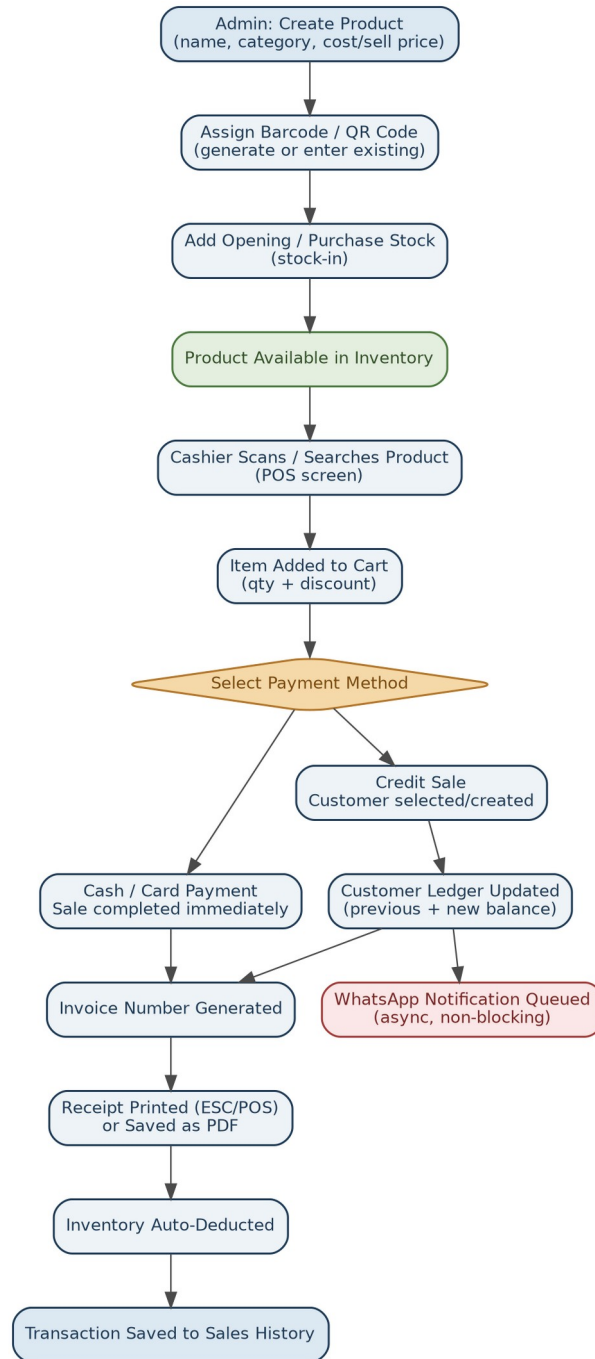


Figure 2 — Product → Barcode → Inventory → Sale → Invoice → Notification flow.

4.3 Deployment Model

- **Server PC:** runs the packaged application, holds the SQLite database file, auto-starts on Windows login, exposes the app over the LAN on a fixed local port.
- **Cashier terminals:** no install required — a browser bookmark pointing at the server's LAN address (a static IP or a friendly hostname, decided at Phase 7) is sufficient.
- **Data:** a single SQLite file on local disk, running in WAL mode for better concurrent read/write behaviour with multiple terminals, plus a scheduled local backup copy.
- **Offline guarantee:** every module except WhatsApp notifications must function with the network adapter disabled — this is a first-class requirement, not an afterthought.

5. Technology Stack & Rationale

Layer	Choice	Why
Language / Runtime	Python 3.11+	Matches the already-agreed stack; mature Windows packaging story via PyInstaller.
Web framework	Flask 3.x	Lightweight, synchronous, simple to reason about for a low-concurrency single-store app; packages cleanly into a single executable.
ORM / migrations	SQLAlchemy 2.x + Alembic (Flask-Migrate)	Type-safe queries, prevents raw-SQL edge-case bugs, and gives a real migration history instead of manual schema edits.
Database	SQLite (WAL mode)	Zero-config, single file, no separate DB server to install/maintain on a shop PC — matches the actual scale of a single store.
Auth	Flask-Login + Werkzeug password hashing	Simple, well-understood session-based auth sufficient for a 2-role (Admin/Cashier) system.
Frontend rendering	Jinja2 server-rendered templates + htmx + Alpine.js for interactivity, styled with Tailwind CSS	Avoids a separate SPA build pipeline and a Node.js dependency inside the shipped app, while still giving every screen a modern, snappy, no-full-reload feel and a contemporary visual design.
Desktop wrapper	pywebview	Turns the Flask app into a native-feeling desktop window without bundling a full Chromium/Node runtime like Electron would.
Packaging	PyInstaller (onedir) + Inno Setup	Produces a real Windows installer with a Start Menu entry, auto-start option, and clean uninstall — no Python installation required on the client PC.
Receipt printing	python-escpos (primary), browser print-to-PDF (fallback)	Direct ESC/POS control gives an unattended, dialog-free receipt print matching real POS hardware behaviour.
Barcode/QR generation	python-barcode, qrcode	Generates printable product labels; scanning itself uses standard HID keyboard-wedge input, needing no special driver.
PDF generation (invoices/reports)	ReportLab	Pure-Python, no external system libraries — far more reliable to bundle inside a PyInstaller Windows executable than alternatives that need Cairo/Pango.
Background jobs / retry queue	APScheduler (in-process)	Runs monthly statements and WhatsApp retry logic without needing a separate broker/service (e.g. Redis+Celery) on a single offline machine.
Testing	pytest, pytest-flask, coverage.py	Standard, well-supported Python testing stack; integrates cleanly with the DoD gates in Section 11.

Layer	Choice	Why
Logging	Python logging, rotating file handler	Local, persistent diagnostic trail for a system that runs unattended on a shop PC.

5.1 UI / Visual Design Layer — A Modern Feel Without Changing the Architecture

This is a deliberate refinement of the frontend rendering row above, not a different architecture. The goal is an interface that feels current — smooth, no full-page reloads, a clean contemporary look — while keeping the single-Flask-app structure, the LAN/browser access model for cashier terminals, and the packaging plan entirely unchanged.

Piece	Role	Why This, Not a Full SPA
htmx	Swaps only the part of the page that changed (e.g. adding an item updates just the cart panel, not the whole screen).	Gets the core "modern app" feel — no flicker, no full reloads — without a JSON API layer or a JS build step.
Alpine.js	Small, in-page client-side reactivity for things like dropdowns, modals, live totals, and form validation.	Same job a heavier framework would do for these interactions, at a fraction of the complexity and with zero build tooling.
Tailwind CSS	Utility-first styling for a clean, contemporary visual design across every screen.	Pure CSS output — compiled once during build, shipped as a static stylesheet. Doesn't touch the backend, the routes, or the packaging.

Practical consequences of this choice:

- Every Flask route still returns HTML, not JSON — no separate API layer to design, version, or test.
- No Node.js runtime is shipped inside the Windows installer — Tailwind's CSS is compiled once at build time by the developer, and only the resulting static .css file is bundled.
- No new packaging step, no new STOP AND ASK gate, and no change to any phase in Section 11 — this only affects how templates are written and styled from Phase 1 onward.
- If a future screen genuinely needs SPA-level richness (e.g. a heavily animated analytics dashboard), that would be a new, separate decision — scoped, grilled, and ADR'd on its own — not a retrofit of this choice.

6. Features — What We're Building, and Why

A. User Access (Basic Admin / Cashier)

Why: every action in a POS needs to be attributable and every sensitive screen needs to be gated, even without a full permissions system.

- Login/logout with hashed passwords, session-based auth.
- Two built-in roles: Admin (full access) and Cashier (POS/billing-only access).
- Cashier routes are hard-blocked from admin-only functions at the server, not just hidden in the UI.

B. Product & Inventory Management

Why: accurate stock is the foundation everything else (billing, reports, low-stock alerts) depends on.

- Categories, product details, SKU, purchase/selling price.

- Stock-in, stock-out, and manual adjustments — each adjustment requires a reason and is logged.
- Low-stock threshold and alert list.
- Full inventory transaction history per product.

C. Barcode / QR Code Management

Why: this is what makes checkout fast — a cashier should rarely need to type a product name.

- Barcode/QR generation and assignment per product.
- Duplicate-code prevention at the database level.
- Scan-to-cart lookup that resolves price, unit, and stock in one step.

D. POS & Billing

Why: this is the daily-use core of the system — it has to be fast, correct, and resilient to real-world interruptions (power blips, offline printer, etc.).

- Search or scan to cart, quantity and discount handling.
- Cash / Card / Credit payment selection.
- Atomic invoice numbering — no duplicate or skipped invoice numbers under concurrent sales.
- Automatic inventory deduction tied to the same transaction as the sale (all-or-nothing).
- Receipt printed via ESC/POS, with PDF as a fallback if no printer is available.
- Returns/refunds flow.

E. Credit Customer Management

Why: many local shops run on trust-based credit, and losing track of who owes what is a real business risk this system removes.

- Customer profiles and credit ledger.
- Full or partial payments against outstanding balance.
- Per-customer statement view and export.

F. WhatsApp Notification Module

Why: automatic, no-effort communication to credit customers on purchase and monthly statements — without ever risking the checkout flow itself.

- Credit-sale notification: items, amount, previous and new balance.
- Monthly statement generation and delivery.
- Provider-agnostic adapter, so the actual WhatsApp API vendor can change without touching business logic.
- Fully asynchronous — a down or slow provider never delays or blocks a sale.

G. Reports & Dashboard

Why: an owner needs to see the state of the business at a glance and pull specific numbers (profit, outstanding credit, low stock) without digging through raw records.

- Sales, inventory, low-stock, outstanding-credit, and basic profit/loss reports.
- Date-range filters and CSV/PDF export.

H. Windows Installable, Offline-Capable Application (New Scope)

Why: the owner should be able to install this like any other Windows program on the shop PC and have it simply be running every morning — not remember to start a server manually.

- PyWebview desktop window wrapping the existing Flask app — no browser chrome, no address bar.
- Single Windows installer (Inno Setup) with Start Menu entry and optional auto-start on login.
- Fully offline for every module except WhatsApp notifications.
- Uninstall preserves the database by default, with an explicit, separate option to remove data.

7. Key Trade-offs & Decisions

These are the decisions already made for this project, recorded here so the agent does not need to re-litigate them — each should still get a corresponding ADR at the relevant grill-with-docs session.

Decision	Chosen	Alternative Considered	Why
Database	SQLite (WAL mode)	PostgreSQL / MySQL	Zero-config, single file, matches single-store scale. Trade-off: limited concurrent-write throughput and a future migration would be needed to scale to multiple stores.
Web framework	Flask (sync)	FastAPI (async), Django	Flask is simplest to package and matches the low-concurrency LAN use case. FastAPI's async model is unneeded complexity here; Django brings more baked-in assumptions than the project needs.
UI architecture	Server-rendered Jinja2 + htmx + Alpine.js + Tailwind CSS, applied across the whole app	Full SPA (React/Vue)	Delivers a genuinely modern look and snappy, partial-page interactivity with no separate JS build pipeline or Node.js dependency to bundle. Trade-off: less suited to highly custom, animated, app-like screens (e.g. a rich analytics dashboard) than a full SPA — acceptable here since no screen in this product needs that.
Desktop wrapper	pywebview	Electron	Much smaller install footprint, reuses the Python runtime already required. Trade-off: depends on the Windows WebView2 runtime being present (pre-installed on modern Windows 10/11; a small risk on older systems).
Receipt output	python-escpos (direct) + PDF fallback	Browser print dialog only	Unattended, dialog-free printing matches real POS hardware UX. Trade-off: needs per-printer-model testing during Phase 7.
WhatsApp delivery	Async queue with retry (APScheduler)	Synchronous call during checkout	Guarantees a slow/down provider can never block or freeze a sale. Trade-off: slightly more moving parts (a queue table, a background worker).

Decision	Chosen	Alternative Considered	Why
Access control	Basic Admin/Cashier with an extensible permissions table underneath	Full configurable RBAC now	Matches the already-agreed scope and avoids building an unscoped module. Trade-off: not future-proof out of the box, but the underlying table shape allows RBAC to be added later without a rewrite.
PDF engine	ReportLab	WeasyPrint	Pure-Python with no external system libraries — reliably bundles inside a PyInstaller Windows executable. WeasyPrint needs Cairo/Pango, which is painful to ship on Windows.
Background jobs	APScheduler (in-process)	Celery + Redis	No extra service/daemon to install on a single offline shop PC. Trade-off: not built for distributed/heavy workloads, which this project does not need.
Packaging mode	PyInstaller onedir	PyInstaller onefile	Onedir starts noticeably faster and is easier to diagnose if something is missing at runtime. Trade-off: ships as a folder rather than a single .exe, handled transparently by the Inno Setup installer.

Recommendation: proceed with every choice above by default. None should be treated as final until its corresponding grill-with-docs session and ADR exist — this table is a starting position for that conversation, not a substitute for it.

8. Living Documentation System

This project maintains three always-current documents under /docs. They are not write-once artifacts — they are updated continuously as work happens.

8.1 /docs/context.md — Single Source of Truth

Everything a new session (human or agent) needs to get oriented in five minutes lives here. It must be updated at the end of every work session, not just at the end of a phase.

- Current phase and its status.
- Summary of decisions made so far (with links to the relevant ADRs).
- Open questions awaiting human approval (mirrors any pending STOP AND ASK gates).
- Known issues / deferred work.
- The full running edge-case list (see Section 10), with a checkbox per case for "has a test."
- A changelog of what happened in each session, in reverse-chronological order.

8.2 /docs/adr/NNNN-title.md — Architecture Decision Records

One file per real decision, numbered sequentially. Template:

```
# NNNN — Title
Status: Proposed | Accepted | Superseded

## Context
```

What problem forced this decision.

Decision

What was chosen.

Alternatives Considered

What else was on the table, and why it lost.

Consequences

What this makes easier, what it makes harder, and what it forecloses.

8.3 /docs/glossary.md — Domain Terms

Plain definitions for domain-specific terms as they are introduced (e.g. stock-in, credit ledger, SKU, WAL mode, invoice number, statement cycle). Refined and expanded during grill-with-docs sessions.

9. Edge Cases & Test Matrix

Every item below must have a corresponding automated test before the owning phase can close. This list itself lives in /docs/context.md and grows as new cases are found.

Inventory

- Stock cannot go negative through normal sale flow.
- Duplicate barcode/QR assignment is rejected at the database level.
- Every manual stock adjustment requires a reason and is logged with who/when.
- Concurrent stock updates from two terminals in the same second resolve without lost updates.
- Deleting a product that has sales history is a soft delete, not a hard delete.

POS / Billing

- Discount cannot exceed the item/cart total.
- Checkout is blocked on an empty cart.
- Invoice numbering has no duplicates or gaps under concurrent checkouts.
- A sale and its inventory deduction commit atomically — a crash mid-sale cannot leave stock deducted without a recorded sale, or vice versa.
- A printer being offline degrades to PDF, and never blocks completing the sale.
- Refunds correctly reverse both the ledger (if credit) and the inventory count.

Credit Customers

- Ledger balance math is correct across mixed sequences of sales and partial payments.
- A customer with an outstanding balance cannot be deleted.
- Duplicate customers (same phone number) are prevented or flagged.
- Overpayment is handled explicitly (credit balance), not silently dropped or negative in a confusing way.

WhatsApp Notifications

- Provider timeout or outage never blocks or delays sale completion.

- Failed sends retry with capped, backed-off attempts, then land in a visible failure log.
- No duplicate notification is sent for the same event.
- Missing or invalid phone number is handled gracefully, with the sale still completing.
- Monthly statement generation is correct across month/timezone boundaries.

Barcode / QR

- Scanning an unregistered code gives a clear "not found" result, not a crash.
- Scanner input (which arrives as fast keyboard input plus an Enter key) cannot leak into the wrong form field.
- Two products cannot share the same code.

Auth

- Session expiry is handled without data loss mid-sale.
- A cashier hitting an admin-only route server-side gets a 403, not just a hidden UI element.

Reports

- A date range with no data renders an empty state, not an error.
- Reports spanning a large dataset remain responsive (pagination or streaming as needed).

Local Network & Offline Operation

- The server auto-starts correctly after a PC restart.
- A locked or unexpectedly closed database file recovers cleanly on next start (WAL/checkpoint behaviour verified).
- A documented backup restore procedure is tested, not just assumed to work.
- The server's LAN address does not silently change under the cashiers' feet (static IP or stable hostname, decided in Phase 7).

Windows Install / Packaging

- Fresh install succeeds on a clean Windows VM with no developer tools present.
- Uninstall removes the application but preserves the database by default.
- Upgrade-install preserves existing data.
- Missing WebView2 runtime is detected and handled (bootstrapped or clearly messaged), not a silent crash.

10. Testing & QA Strategy

Test Layer	Tooling	What It Covers	Who Runs It
Unit tests	pytest	Business logic in isolation — ledger math, invoice numbering, stock deduction, discount rules.	Agent, automatically, every commit.
Integration tests	pytest-flask + temp SQLite DB	Full request/response flow through real routes — e.g. a complete sale from cart to saved transaction.	Agent, automatically, per phase.
Mocked-external tests	pytest + mocked provider adapter	WhatsApp success/failure/retry behaviour without a live API.	Agent, automatically, Phase

Test Layer	Tooling	What It Covers	Who Runs It
			5+.
Hardware-in-loop tests	Manual checklist (cannot be automated)	Real barcode scanner HID input and real ESC/POS printer output.	Human, using an agent-generated checklist.
End-to-end / UAT	Scripted full-journey run on a clean Windows VM with the installer	The entire product as the client will actually use it.	Human, with agent-prepared test script.

How the Agent Verifies a Phase Is Actually Done

- Run the full pytest suite; the phase cannot close on red, skipped, or newly-added-but-unrun tests.
- Enforce a minimum coverage threshold on business-logic modules (not routes/templates) — recommend 80%+, to be confirmed at Phase 0's grill session.
- Run a linter/formatter (e.g. ruff) with zero errors.
- Re-check every edge case in Section 10 that belongs to this phase's modules and confirm each has a passing test.
- Update context.md with the phase's outcome, and write/update any ADRs triggered by decisions made in the phase.
- Present the phase's Definition of Done checklist explicitly to the human before starting the next phase — not just internally self-report it.

11. Development Phases

Phases are sequential. A phase does not start until the previous phase's Definition of Done is fully checked and, where flagged, explicitly approved by the human.

Phase 0 — Project Setup & Grill Session

Objective: Establish the repo, the documentation system, and lock the initial data model and module boundaries through a grill-with-docs session before any feature code is written.

Steps

1. Ask the human to run the grill-with-docs skill to interrogate the proposed data model, module boundaries, and glossary.
2. Initialize the repository structure, virtual environment, and base Flask app skeleton.
3. Set up configuration handling (environment-based, no hardcoded secrets) and logging.
4. Set up the pytest scaffold and a local test-runner script.
5. Create /docs/context.md, /docs/adr/, and /docs/glossary.md.
6. Write ADR-0001 recording the overall stack choice (Section 5) and ADR-0002 recording the initial data model shape.

Required Tests

- Empty test suite runs green in CI-less local execution.
- A smoke test confirms `flask run` serves a placeholder page.

● STOP AND ASK — Before closing Phase 0

- Confirm the draft database schema before it becomes the basis for Phase 1 migrations.
- Confirm the final module list (no additions/removals) before coding begins.

Definition of Done — Checklist

- ☐ Repository boots locally from a clean checkout with documented setup steps.
- ☐ context.md, /docs/adr/, and glossary.md exist and are populated with initial content.
- ☐ grill-with-docs session completed; ADR-0001 and ADR-0002 written and marked Accepted.
- ☐ Module list and top-level data model confirmed by the human.

How the Agent Verifies This Phase

- Run the smoke test and confirm a clean start with no errors.
- Confirm the two ADRs exist and are referenced from context.md.

Phase 1 — Core Data Model & Auth

Objective: Implement the underlying tables and the basic Admin/Cashier authentication layer that every later phase depends on.

Steps

1. Define SQLAlchemy models: Product, Category, StockMovement, User, Customer, Sale, SaleItem, Payment, CreditLedgerEntry, NotificationQueue.
2. Generate the initial Alembic migration and confirm it runs cleanly from an empty database.
3. Implement login/logout with hashed passwords and session-based auth for Admin and Cashier roles.
4. Implement server-side route protection so Cashier accounts cannot reach Admin-only endpoints.
5. Write a seed script for local development/testing data.

Required Tests

- Model-level constraint tests (unique SKU, unique barcode, required fields).
- Auth flow tests: successful login, failed login, logout, session expiry.
- Authorization tests: a Cashier session hitting an Admin route returns 403.
- Migration up/down test from an empty database.

● STOP AND ASK — Before closing Phase 1

- Confirm the final field list per table before this migration is treated as frozen — later changes must go through a new migration, not an edit to this one.

Definition of Done — Checklist

- ☐ All models pass their unit tests.
- ☐ Login/logout verified for both roles.
- ☐ Cashier-blocked-from-admin-route test passes.
- ☐ Migrations are reproducible from an empty database with no manual steps.

How the Agent Verifies This Phase

- Run the model and auth test suites; all green.
- Manually attempt an Admin-only action from a Cashier session and confirm it is refused.

Phase 2 — Inventory & Product Management

Objective: Build product/category management, barcode/QR assignment, stock movements, and low-stock alerting.

Steps

1. CRUD for categories and products.
2. Barcode/QR generation and assignment, with duplicate-code prevention.
3. Stock-in, stock-out, and manual adjustment flows, each requiring a reason and producing an audit log entry.
4. Low-stock threshold configuration and an alert list view.

Required Tests

- Negative-stock prevention under normal sale-triggered deduction.
- Duplicate barcode/QR rejection.
- Adjustment audit trail correctness (who, when, reason, before/after quantity).
- Low-stock alert fires exactly at the configured threshold boundary.

Definition of Done — Checklist

- ☐ Full CRUD covered by tests, including the edge cases above.
- ☐ Barcode/QR assignment and lookup verified end-to-end.
- ☐ Low-stock alert list verified against seeded data.

How the Agent Verifies This Phase

- Run the inventory test suite; all green.
- Manually simulate a scan (keyboard input) against a seeded barcode and confirm correct product resolution.

Phase 3 — POS & Billing

Objective: Build the cashier-facing sale flow: cart, payment, invoicing, receipt printing, and inventory deduction — the highest-traffic and highest-risk part of the system.

Steps

1. Build the POS screen: search/scan to cart, quantity and discount handling.
2. Implement Cash/Card/Credit payment selection.
3. Implement atomic, race-safe invoice numbering.
4. Tie inventory deduction to the same transaction as the sale (all-or-nothing commit).
5. Implement ESC/POS receipt printing via python-escpos with a PDF fallback.
6. Implement a returns/refund flow that reverses both stock and, where applicable, the credit ledger.

Required Tests

- Concurrent-checkout race test on invoice numbering (no duplicates or gaps).
- Cart edge cases: empty-cart checkout blocked, discount exceeding total blocked.
- Transaction atomicity test: simulate a failure mid-sale and confirm no partial state is left behind.
- Printer-offline fallback test: sale completes and produces a PDF receipt when no printer responds.
- Refund flow test: stock and ledger both reverse correctly.

● STOP AND ASK — Before closing Phase 3

- Confirm discount rules (max discount %, who can apply it) and refund policy before this logic is locked in — both directly affect the ledger and stock correctness.

Definition of Done — Checklist

- ☐ A full sale journey (search/scan → cart → payment → invoice → receipt → stock update → saved transaction) passes as one integration test.
- ☐ Receipt output verified against a real or simulated ESC/POS printer.
- ☐ Refund flow independently tested end-to-end.

How the Agent Verifies This Phase

- Run the full POS integration suite; all green.
- Manually run one real cash sale and one real credit sale on a test machine, and inspect the resulting DB state and printed/PDF receipt.

Phase 4 — Credit Customer Management

Objective: Build customer records, the credit ledger, and statement generation.

Steps

1. Customer CRUD, including duplicate-phone-number handling.
2. Credit ledger entries tied to sales and payments.
3. Full and partial payment application against outstanding balance.
4. Per-customer statement view and export.

Required Tests

- Ledger balance correctness across mixed sale/payment sequences.
- Deletion of a customer with an outstanding balance is blocked.
- Overpayment produces an explicit credit-balance state, not a silent or confusing negative.

Definition of Done — Checklist

- ☐ Ledger reconciles to zero across a full scripted test scenario (sales + partial + full payments).
- ☐ Statement output matches an expected fixture for a known scenario.

How the Agent Verifies This Phase

- Run the credit-management test suite; all green.
- Manually verify one statement export against hand-calculated expected values.

Phase 5 — WhatsApp Notification Module

Objective: Build the provider-agnostic, fully asynchronous notification system for credit sales and monthly statements.

Steps

1. Run a grill-with-docs session to finalize the provider choice, message templates, and retry policy — write the corresponding ADR before coding.
2. Implement a provider adapter interface so the underlying WhatsApp API vendor is swappable.
3. Implement an async queue (APScheduler-backed) that a credit sale enqueues into, rather than calling the provider inline.
4. Implement retry with capped, backed-off attempts and a visible failure log.
5. Implement the monthly statement generation job.

Required Tests

- Mocked-provider tests for send success, send failure, and retry-cap behaviour.

- Explicit test that a sale completes normally with the provider deliberately disabled/unreachable.
- Duplicate-send prevention test for the same event.
- Month-boundary/timezone test for statement generation.

● **STOP AND ASK — Before closing Phase 5**

- Confirm the actual WhatsApp provider/vendor, message template wording, and retry/backoff limits before implementation — this is exactly the kind of decision the grill session above must lock down.

Definition of Done — Checklist

- ☐ Sale flow is verified unaffected by provider downtime — this is a hard gate, not a nice-to-have.
- ☐ Retry and failure-logging behaviour verified against the capped policy from the ADR.
- ☐ ADR for the chosen provider adapter and message templates exists and is Accepted.

How the Agent Verifies This Phase

- Run the notification test suite with the provider mocked as both available and unavailable; both must pass.
- Manually trigger one real send (if a live provider/sandbox is available) and confirm delivery and logging.

Phase 6 — Reports & Dashboard

Objective: Build the reporting and dashboard views owners actually use to run the business.

Steps

1. Sales, inventory, low-stock, outstanding-credit, and basic profit/loss reports.
2. Dashboard summary view with the most-used numbers at a glance.
3. Date-range filters and CSV/PDF export.

Required Tests

- Empty-range handling (no data in the selected window renders cleanly).
- Date/timezone boundary correctness.
- Performance check against a seeded large dataset (thousands of transactions).

Definition of Done — Checklist

- ☐ Every report type verified against known seeded data with hand-checkable expected totals.
- ☐ Export (CSV/PDF) verified to open correctly and match on-screen figures.

How the Agent Verifies This Phase

- Run the reporting test suite; all green.
- Manually spot-check one report's numbers against a manually tallied sample.

Phase 7 — Windows Packaging & Offline Installable App

Objective: Turn the working Flask application into a proper, offline-capable Windows install, with the desktop-window experience layered on top.

Steps

1. Run a grill-with-docs session to finalize the installer tool, auto-start method, database file location, and update strategy — write the corresponding ADR.
2. Wrap the Flask app with pywebview for a native-feeling desktop window.

3. Build the PyInstaller (onedir) bundle.
4. Author the Inno Setup installer script: Start Menu entry, optional auto-start on login, database file under a stable per-machine location, and an uninstall flow that preserves data by default.
5. Add a WebView2 runtime presence check with a clear fallback/bootstrap path.
6. Decide and implement the LAN addressing approach for cashier terminals (static IP or stable hostname).

Required Tests

- Fresh install test on a clean Windows VM with no developer tools present.
- Uninstall test confirming the database is preserved by default.
- Upgrade-install test confirming existing data survives.
- Auto-start-after-reboot test.
- Full offline smoke test of every non-WhatsApp module with the network adapter disabled.

● STOP AND ASK — Before closing Phase 7

- Confirm the exact database file location, backup file location, and whether auto-start-on-login is on by default before the installer script is finalized — these are painful to change after client machines have real data in them.

Definition of Done — Checklist

- ☐ Installer runs start-to-finish on a clean VM without any developer environment present.
- ☐ Uninstall/upgrade behaviour verified exactly as specified above.
- ☐ Offline smoke test passes in full with networking disabled.
- ☐ ADR for the packaging and update-strategy decisions exists and is Accepted.

How the Agent Verifies This Phase

- Execute the clean-VM install/uninstall/upgrade test script and record the results.
- Disable networking on the test VM and run the full manual smoke-test checklist across every module.

Phase 8 — Full System Integration Testing & UAT

Objective: Prove the whole system — every module, on the real packaged installer, under realistic multi-terminal and hardware conditions.

Steps

1. Run a full scripted journey covering every module in sequence on the packaged installer.
2. Run a multi-terminal LAN test with two or more browser clients hitting the server simultaneously.
3. Run the hardware-in-loop checklist with a real barcode scanner and real thermal printer.
4. Run a backup/restore drill.
5. Run a soak test simulating an extended, high-volume business day of transactions.

Required Tests

- Every edge case in Section 10 is individually re-confirmed as covered by a passing test.
- All prior phase Definitions of Done are re-verified together, not just individually at the time.

● STOP AND ASK — Before closing Phase 8

- Present the completed sign-off checklist and explicitly request human go-live approval before considering the project complete — this is the final gate of the entire project.

Definition of Done — Checklist

- ☐ Every item in Section 10's edge-case matrix is checked off.
- ☐ context.md is fully up to date and every ADR is in an Accepted (or explicitly Superseded) state.
- ☐ A final sign-off checklist is presented to the human for go-live approval.

How the Agent Verifies This Phase

- Run the complete automated test suite across the whole codebase; all green.
- Walk through the hardware-in-loop and soak-test checklists and record results in context.md.
- Present the final Definition of Done (Section 12) to the human explicitly and wait for go-live approval.

12. Final "Ready for Production" Definition of Done

The project is not complete until every item below is true simultaneously, not just true at some point during development.

- ☐ Every phase's individual Definition of Done (Section 11) is checked and was explicitly approved at its gate.
- ☐ Every edge case in Section 10 has a passing automated test, except hardware-in-loop items, which have a passing manual test recorded in context.md.
- ☐ The full automated test suite is green with no skipped tests.
- ☐ The Windows installer has been verified on a clean VM: install, uninstall (data preserved), and upgrade (data preserved).
- ☐ A full offline smoke test (network disabled) passes across every module except WhatsApp notifications.
- ☐ WhatsApp notifications have been verified both to succeed and to fail-gracefully without ever blocking a sale.
- ☐ context.md accurately reflects the current, final state of the system with no open questions remaining.
- ☐ Every ADR is in an Accepted or explicitly Superseded state — none left as Proposed.
- ☐ The glossary is complete for every domain term used in the codebase and documentation.
- ☐ A human has explicitly signed off go-live in context.md.

13. Agent Kickoff — Start Here

This is the literal first-session instruction sequence. Follow it in order.

1. Read this entire document once, fully, before writing any code.
2. Check whether /docs/context.md already exists. If it does, read it fully — it is more current than this document for anything it covers. If it doesn't, create it using the structure in Section 8.1.
3. Tell the human it is time to run the grill-with-docs skill for Phase 0, and wait — do not begin implementation before this session and its ADRs exist.
4. Once Phase 0's grill session is complete and its ADRs are written, proceed through the phases in Section 11, in order, without skipping any.
5. At every  STOP AND ASK gate, stop, present the decision clearly with a stated recommendation, and wait for explicit human approval before continuing.
6. At the end of every work session (not just every phase), update context.md with what happened, what's open, and what's next.

7. When a phase's Definition of Done is fully checked, present it explicitly to the human before starting the next phase.
8. If at any point a new decision would contradict an existing ADR, stop and flag it — do not silently override a prior decision.

End of specification. This document, together with /docs/context.md, /docs/adr/, and /docs/glossary.md as they are created, constitutes the complete and current source of truth for this project.